Verified Stopping, Not Adaptive Allocation, Saves Reads in Multi-Container DNA Storage: A Leak-Free Computational Study on a Digital Twin and Real Read Streams

Sai Mouli
saimouli12345678@gmail.com

***Abstract*—When an erasure-coded DNA archive is spread over several separately synthesised and handled containers of unknown quality, two retrieval decisions arise: when to stop sequencing and which container to read next. We find that the first matters and the second, as usually modelled, does not. All work is computational; nothing was synthesised or sequenced. On the DT4DDS digital twin with six independently synthesised, aged and amplified containers, a streaming decoder that stops only when every stripe, its CRC-32 and the file's SHA-256 verify used on average 27% fewer reads than the best hindsight-tuned static plan must reserve (3.49k vs. 4.80k), with no model at all. Model-based container selection, including Thompson sampling, UCB, learn-then-commit and information-greedy rules, was a measured negative when one container was near-dead ($+7\%$ to $+441\%$ reads against round-robin), because one efficiency multiplier cannot represent dropout; it helped only after an outright container failure (up to -24%) or under mild heterogeneity (-2 to -4%). No rule lowered the 0.99-quantile. Over 23,160 end-to-end retrievals with no oracle, no output file was wrong (exact 95% upper limit 1.6×10^{-4}) and no wrong payload passed the inner checks in 30.2 million decodes (bound 1.2×10^{-7}). A leak-free replay of real Illumina read streams from two vendors (secondary, oracle decodability) agrees: stopping cut reads against fixed depth by 14% and 25%, and allocation with a generic prior changed them by at most 3%; only a prior calibrated on held-out reads of the same containers, which identified a near-dead container in advance, saved more (16% on one pool). Every number is regenerated from committed result files under a frozen protocol with logged deviations, and sampled retrievals re-run bit for bit.**

***Index Terms*—DNA data storage, adaptive sequencing, verified stopping, erasure coding, coverage depth, digital twin, Bayesian calibration, multi-armed bandits, reproducibility**

I. Introduction

DNA has been demonstrated as an archival medium at kilobyte to hundreds-of-megabyte scale [1]–[5]. Retrieval requires sequencing, which samples molecules at random: some strands are read many times, others never, and reads carry substitution, insertion and deletion errors [6], [7]. Systems therefore protect each strand with an inner code and groups of strands with an outer erasure code, and sequence to a fixed depth expected to recover enough strands [3], [5]. The number of reads needed for decoding—the *coverage depth*—is now studied in its own right [8], [9].

We consider archives split across several physically separate *containers* (vials, capsules, pools) that are synthesised, stored and handled separately. This is a working assumption rather than established practice; random access so far addresses files within one pool [5], [10], and recent theory treats erasure-coded data partitioned over containers with uniform sampling [11] and container failures [12]. Containers age, are amplified and fail differently, and these differences are unknown when a retrieval starts. Two operational questions follow: *when* to stop sequencing, and *which* container to sequence next.

Problem. Given an erasure-coded file distributed over M containers of unknown, heterogeneous quality, some of which may have failed, sequence in batches and decide after each batch which container to read and whether to stop, so that the output file is byte-identical to the stored one with high probability while the number of reads is small.

Findings. (i) *Verified stopping is the robust saving*: it needs no model, never returned a wrong file, and lowered the mean reads per retrieval against fixed-depth sequencing in every setting (digital twin and both real-read pools) and against the hindsight-tuned static plan on the digital twin and the Twist replay; on the Genscript replay the hindsight plan, which knows in advance to avoid a near-dead container, was 1% cheaper. (ii) *Adaptive allocation is a measured negative* when a container is near-dead: every model-based rule we tried read more than round-robin; it helps only after an outright failure, under mild heterogeneity, or when calibration data identify the weak container before retrieval. (iii) No rule lowers the 0.99-quantile, so the gains serve pay-per-read users, not users who reserve capacity. (iv) Two results mark the *design boundaries* of the archive rather than of the policies: with two failed containers or with a noisier (photolithographic) synthesis model, the file is unrecoverable by any policy.

Scope. All work is computational. Nothing was synthesised or sequenced for this paper, and no result should be read as a wet-lab demonstration. We use the term *exact retrieval* only for our digital-twin pipeline, where the output file is checked byte for byte against the stored file; on replayed real reads, decodability is an oracle event and we speak of *recovery*.

Contributions.

1) *Evidence that verified stopping is the robust saving.* A verification-gated streaming retrieval pipeline that stops only when every stripe, its CRC-32 and the file's SHA-256 verify (Section IV), evaluated against a static family that includes a hindsight-tuned plan. Stopping when decodable [13] and checksum-verified decoding [4] are known; quantifying their value for multi-container retrieval is, to our knowledge, new.
2) *A measured negative for adaptive allocation.* An online-adaptive controller family (a discrete posterior over container efficiency with a failure atom and a beta-binomial likelihood, and eight rules including Thompson sampling, UCB, learn-then-commit and information-greedy design [14]–[17]) loses to round-robin under a near-dead container; we identify the cause (a single efficiency multiplier cannot represent dropout), map where allocation helps (outright failure, mild heterogeneity) with a severity sweep, and show that the better-calibrated likelihood allocates worse.
3) *Leak-free, end-to-end reproducibility and safety bounds.* The full chain from DT4DDS reads [7] of independently synthesised containers to bytes, with no oracle, under a protocol frozen before the final runs; exact bounds on wrong outputs and false accepts; a leak-free re-analysis of real Illumina streams, with our earlier leaky analysis kept as an upper bound (Appendix A); bit-for-bit re-runs and a claim-to-evidence map (Section IX).

II. Background and Related Work

DNA storage systems and channels. Storage systems combine constrained encodings with inner and outer codes [2]–[5], [18], [19]. Coverage is over-dispersed, a few percent of strands drop out, and errors arise mostly in synthesis and sequencing [6], [7], [20]. DT4DDS [7] is a digital twin of the synthesis, aging, PCR and sequencing steps whose error and bias parameters were fitted to experiments; we use it as the simulator of our primary evidence. Information-theoretic models treat the channel as sampling with replacement followed by noise [21]–[23].

Coverage depth. For a sampler that cannot be controlled, MDS codes and uniform sampling minimise the expected number of reads [8]. Follow-ups study random access [9], real-data sampling distributions [24], partitioning over containers [11], container failures [12] and the reliability of MDS-coded retrieval [25]. These works analyse *fixed* sampling laws; we study a *controllable* sampler.

Adaptive sequencing and sequential decisions. Nanopore adaptive sampling selects molecules in real time [26]–[28], and DNA-storage readouts have been sequenced until sufficient coverage [13]. Choosing a container resembles a multi-armed bandit with a combinatorial (K-of-N per stripe) reward; we use Thompson sampling [14], [29], upper confidence bounds [15], explore-then-commit [16] and an information-greedy rule from Bayesian experimental design [17] as baselines (see [30]). We prove no regret bound.

Placement and consensus. Erasure-coded storage limits the fragments of a stripe per failure domain [31]–[35]. Strands are reconstructed from noisy reads by trace reconstruction [36]–[38] after clustering [39] or learned models [40]; systematic nanopore errors are documented [41], [42]. Table I positions this work.

III. System Model

Table II lists the notation.

A. Storage

A file is bundled, hashed (SHA-256) and split into S stripes of K data fragments of P bytes. A systematic Cauchy Reed–Solomon code [45] over GF(2^8) extends each stripe to N fragments, any K of which suffice. Each fragment becomes a strand carrying an address, an address check byte, the payload and an inner Reed–Solomon code with four parity bytes plus a CRC-16, mapped to DNA by a whitened 2-bit code. Code parameters, a CRC-32 per stripe and the SHA-256 are kept in a manifest outside the DNA. Fragments are assigned round-robin to M containers.

B. Sequencing and recovery model used by the controller

Reads from container j form a stream. The controller's model (used only to rank containers and to forecast; never to decide that a fragment is recovered) treats the reads of a stored fragment as negative-binomial with mean $\theta_j\, R_j\, \bar{\pi}_j$ after R_j reads, where $\bar{\pi}_j$ is the prior per-fragment read share (yield over stored fragments) and θ_j an unknown efficiency multiplier (θ_j=0: failed container). A fragment becomes decodable after t reads with probability $\varphi(t)$ (consensus-success curve), and a molecule is absent with probability d (dropout). Integrating over abundance gives the probability $Q(\mu)$ that a stored fragment is decodable at mean coverage μ:

$$Q(\mu) = (1-d)\,\mathbb{E}_g\Big[\sum_{n\geq 0} \mathrm{NB}(n;k,g\mu)\,\varphi(n)\Big], \qquad (1)$$

with a shared log-normal strand effect g (Gauss–Hermite quadrature). Every container starts from the same generic literature prior (yield 0.9, NB shape k=10, dropout 0.5%, a generic consensus curve, failure probability 10^{-3}); no calibration data of the evaluated system is used in the primary experiments.

C. Objective and who benefits

A policy chooses, after each batch of b reads, a container and whether to stop. We report two costs because they serve different users. A *pay-per-read* user (e.g. streaming nanopore sequencing, or a service that bills reads) pays the reads actually used: the relevant metric is the *mean* reads per retrieval. A user who must *reserve* capacity in advance (e.g. a fixed flowcell) pays for the 0.99-quantile (q99) on every retrieval. A static plan, which commits read proportions and a budget in advance, pays its q99 every time; its mean cost *is* its q99. Adaptive stopping helps the first user even when it does not reduce q99.

TABLE I

POSITIONING AGAINST THE CLOSEST PRIOR WORK. Y: YES; N: NO; P: PARTIAL; I: IMPLICIT (EXPECTED READS UNTIL DECODABILITY ANALYSED, NO OPERATIONAL STOPPING RULE); –: NOT STATED. "CONTAINERS" ARE STORAGE NODES IN [35] AND SAMPLING SOURCES IN [43].

Work	Multi-container	Container failures	Heterog. containers	Correlated errors	Adaptive read allocation	Stopping rule	Real reads	End-to-end digital twin
Lopez et al. 2019 [13]	N	N	N	–	N	P	Y	N
Gimpel et al. 2023 [7]	N	N	P	N	N	N	Y	Y
Bar-Lev et al. 2025 [8]	N	N	N	N	N	I	N	N
Gruica et al. 2024 [9]	N	N	N	N	N	I	N	N
Cao & Chen 2025 [24]	N	N	N	N	N	I	P	N
Kas Hanna 2026 [25]	N	N	N	N	N	N	N	N
Levy et al. 2026 [12]	Y	Y	N	N	N	I	N	N
Kong et al. 2026 [11]	Y	N	P	N	N	I	N	N
Leong et al. 2012 [35]	Y	Y	N	N	N	N	N	N
Weilguny et al. 2023 [28]	N	N	–	N	Y	–	Y	N
Bubeck et al. 2013 [43]	Y	N	Y	N	Y	N	N	N
This work	Y	P	Y	P	Y	Y	P	Y

Notes on "this work": container failures are simulated (digital twin) or injected (replay), not observed; correlated errors are validated on real nanopore reads but enter the controller only through consensus curves; real reads (ENA PRJEB65931 [44], same-batch aliquots) are used in a semi-empirical replay with oracle decodability, so they are marked partial; the end-to-end digital twin is DT4DDS [7]. Gimpel et al. [7]: heterogeneity characterised across handling conditions of one pool, and the digital twin itself. Cao & Chen [24]: log-normal channel fitted to their own data. Weilguny et al. [28]: genomic loci, not storage containers. Kong et al. [11]: cells based on the abstract only.

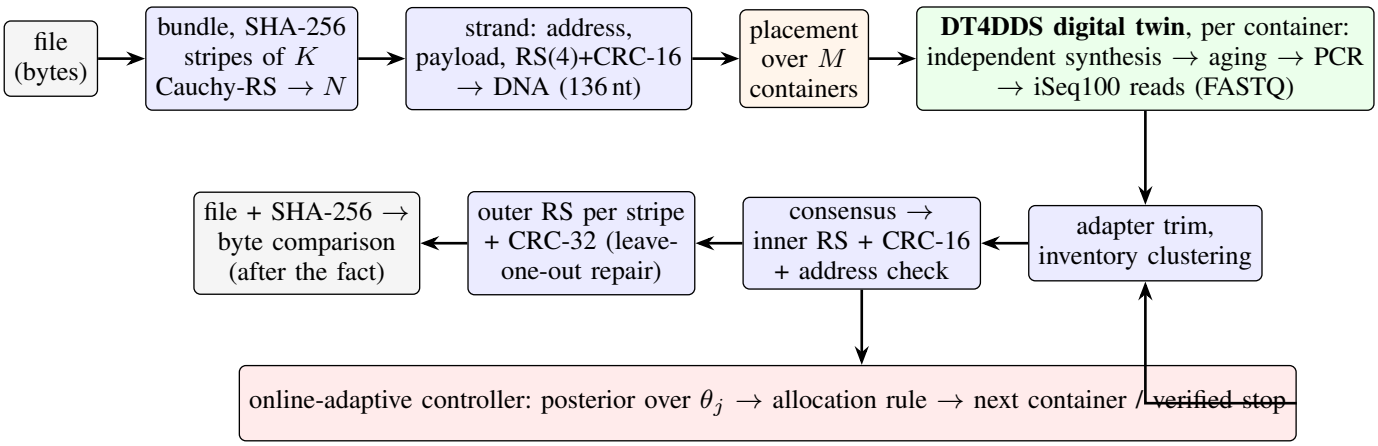


Fig. 1. Pipeline of the primary experiments (conceptual diagram). Every container is an independent DT4DDS synthesis with its own handling history; its reads are consumed in batches by the same streaming decoder that reads FASTQ files. The true file is used only after a retrieval ends, to count wrong outputs and false accepts.

IV. ONLINE-ADAPTIVE RETRIEVAL

A. Posterior over container efficiency

The support of θ_j is $\Theta = \{0\} \cup \{\theta^{(1)}, \ldots, \theta^{(25)}\}$, log-spaced in $[0.01, 5]$, with prior $p_0(0) = f_j$ and $p_0(\theta^{(\ell)}) \propto (1 - f_j)\exp(-\frac{1}{2}(\ln \theta^{(\ell)}/s)^2)$, with $s = 0.5$ in every policy run. After R_j reads with U_j verified fragments among the n_j stored in container j, we use a beta-binomial likelihood with intra-container correlation ρ_U:

$$p(\theta \mid R_j, U_j) \propto p_0(\theta)\,\mathrm{BB}\big(U_j;\, n_j,\, Q(\theta R_j \bar{\pi}_j),\, \rho_U\big), \qquad (2)$$

where $\mathrm{BB}(u; n, p, \rho)$ has mean np and variance $np(1-p)(1+(n-1)\rho)$; $\rho_U{=}0$ recovers the binomial. Recoveries of a container's fragments are correlated (shared synthesis bias, dropout), so the binomial is over-confident; ρ_U is chosen on disjoint calibration replicates (Section V-D). We call the controller *online-adaptive*, not self-calibrating: it adapts to observations, but its uncertainty is not well calibrated (Section VI-F).

B. Allocation rules

For an unrecovered fragment i in container j, the hazard of becoming decodable within the next b reads is $h_{ij} = [Q(\tilde{\theta}_j(R_j{+}b)\bar{\pi}_j) - Q(\tilde{\theta}_j R_j \bar{\pi}_j)]/[1 - Q(\tilde{\theta}_j R_j \bar{\pi}_j)]$, where $\tilde{\theta}_j$ is chosen by the rule:

- *Round-robin + verified stop*: reads every container in turn (no model).
- *Coverage-greedy*: $\tilde{\theta}_j$ = posterior mean; reads $\arg\max_j \sum_i h_{ij}$ over fragments of unfinished stripes.
- *Pivotal*: weights each fragment by a normal-approximation sensitivity of its stripe's completion probability (our earlier rule; details in the supplement).
- *Thompson sampling* [14], [29]: $\tilde{\theta}_j$ drawn from the posterior; coverage score.
- *UCB* [15]: $\tilde{\theta}_j$ = posterior 0.9-quantile.
- *Learn-then-commit* [16]: five round-robin batches per container, then the posterior is frozen and the coverage score used.

TABLE II
NOTATION (CODE OBJECTS IN PACKAGE D3RA).

Symbol	Meaning	Code
K, N, S, P	data / total fragments per stripe, stripes, payload bytes	`erasure_codes`
M, j	number of containers, container index	`Layout`
n_j	fragments stored in container j	`Layout`
b	batch size (reads per decision)	`AdaptiveConfig`
R_j, U_j	reads taken from j, verified fragments from j	`StreamRetriever`
θ_j, Θ, s	efficiency multiplier, its grid, prior log-sd	`AdaptiveConfig`
f_j	prior probability that container j failed	`ChannelParams`
$\bar{\pi}_j$	prior per-fragment read share of container j	`Adaptive Controller`
k, d, $\varphi(t)$	NB shape, dropout, consensus-success curve	`ChannelParams`
$Q(\mu)$	P(fragment decodable) at mean coverage μ	`_QTable`
ρ_U	beta-binomial ICC of recoveries in the likelihood	`overdispersion`
h_{ij}	hazard of recovery within the next batch	`scores`
q	posterior quantile for UCB (q) and tail-aware ($1-q$)	`quantile`
ν_s	remaining need of stripe s (K minus verified)	`scores`
w_j, e_j, α	static proportions, hindsight per-read recovery rate, exponent	`static_family`
θ_j^*	calibration target (best-fitting multiplier)	`_theta_star`
q99	0.99-quantile of reads per retrieval	`dt_analysis`

- *Info-greedy* (Bayesian experimental design [17]): reads the container with the largest posterior variance of $\ln \theta_j$ until all fall below 10^{-3}, then coverage-greedy.
- *Tail-aware*: $\tilde{\theta}_j$ = posterior 0.1-quantile (pessimistic) and the score counts only the 10% of stripes with the largest remaining need (the bottleneck stripes that set the retrieval's length); a heuristic aimed at the q99.

All rules probe each container with one batch first. The *static* family reads fixed proportions $w_j \propto e_j^{\alpha}$, $\alpha \in \{0, 0.5, 1, 2\}$, until decodable, where e_j is the per-read recovery rate of container j measured in hindsight on the same scenario: $\alpha{=}0$ is uniform fixed depth, $\alpha{=}1$ equalises expected coverage, and the best α is a hindsight-tuned ("oracle") static plan.

C. Verified stopping

A retrieval stops with an output file only when (i) every stripe holds K fragments that each passed inner RS decoding, CRC-16 and the address check, (ii) every stripe matches its CRC-32, and (iii) the file matches its SHA-256. If (ii) fails, one more fragment is requested for that stripe and the faulty fragment is localised by leave-one-out decoding. If the streams are exhausted first, the retrieval reports failure; it never returns an unverified file. An early-abandonment (futility) rule is disabled in all runs.

D. Completion forecast

The controller also forecasts the probability of finishing within a remaining budget. The *analytic* forecast multiplies per-stripe normal approximations of $\mathbb{P}(\sum_{i \in s} X_i \geq \nu_s)$ with the remaining budget spread evenly over containers. The *posterior-simulation* forecast (default) draws 200 values of θ from the posterior, spreads the remaining budget in proportion to the reads spent so far, simulates per-fragment recoveries and reports the fraction of draws in which every stripe completes.

V. EVALUATION METHODOLOGY

A. Digital-twin end-to-end pipeline (primary evidence)

We store a fixed 18,432-byte random file with $P{=}24$-byte payloads, $(K, N) = (32, 48)$, 25 stripes and 1,200 strands of 136 nt over $M{=}6$ containers. For each container we run DT4DDS 1.1.0 [7] with its published default parameters: array synthesis (Twist model; log-normal oligo counts, $\sigma{=}0.3$), sampling of a given number of copies per strand, aging, high-fidelity PCR and Illumina iSeq100 single-end 150 nt sequencing, producing 60 reads per stored strand. The six containers are *independent* syntheses with their own seeds and histories. The reads, after adapter trimming, go through inventory clustering against the expected address prefixes, consensus (at most 24 reads per cluster), inner decoding, outer decoding with CRC-32 repair and SHA-256 verification, in batches of 50 reads. Every container decodes its own clusters; nothing uses the stored strands to decide recovery. The stored file and payloads are used only after the retrieval to count wrong outputs and false accepts (a wrong payload that passed the inner checks).

B. Scenarios

Baseline: fresh; aged one half-life; aged two half-lives with 10 copies per strand; 30 PCR cycles; 45 PCR cycles with 20 copies; near-dead (two half-lives, 40 cycles, $\sigma{=}0.5$, 3 copies). *One/two failed*: the fresh (and the 30-cycle) container returns only non-informative reads. *Homogeneous control*: six fresh containers. *Photolithographic*: the baseline histories with DT4DDS's photolithographic synthesis model. *Severity sweep*: the baseline with the near-dead container's copy number set to 200, 30, 10, 3 or 1.

C. Frozen protocol and statistics

Design, policies, metrics, statistics, pre-specified comparisons and the calibration rule were written to `dt_protocol.yaml` before the final runs; deviations are listed in Appendix B. Each scenario uses 30 independent syntheses of all six containers $\times$ 10 read orders (300 retrievals per policy); seeds are derived with CRC-32 of the scenario, replicate and container, so every run is bit-for-bit reproducible. CIs are cluster bootstrap percentile intervals over syntheses (2,000 resamples [46]); comparisons are paired over the same syntheses and read orders; wrong-output and false-accept rates carry exact Clopper–Pearson bounds [47]. We report every pre-specified comparison, including negative ones, and no significance stars.

D. Calibration protocol

ρ_U is chosen from $\{0, 0.003, 0.01, 0.03, 0.1\}$ on ten *disjoint* baseline replicates (seeds 1000–1009) as the value whose 90% credible interval covers the target θ_j^* closest to 90% of the time. θ_j^* is the multiplier that best fits the container's full recovery curve under the generic model, and the selection used prior sd $s{=}1$ whereas the policy runs use $s{=}0.5$ (a mismatch we noticed after freezing and report rather than retune); intervals are evaluated at the read count where the prior

TABLE III
SAFETY OF VERIFICATION-GATED STOPPING OVER EVERY DIGITAL-TWIN RETRIEVAL RUN FOR THIS PAPER (ALL SCENARIOS, POLICIES, STATIC PLANS AND ABLATIONS; THE FORECAST-EVALUATION RUNS ARE EXCLUDED). UPPER LIMITS OF EXACT TWO-SIDED 95% CLOPPER–PEARSON INTERVALS.

End-to-end retrievals	23,160
output byte-identical to stored file	16,032
reported as not recovered (no output)	7,128
wrong output (file returned, not identical)	0
Wrong-output rate, 95% upper bound	1.6e-04
Fragment decode attempts	30,164,044
false accepts (wrong payload passed inner checks)	0
False-accept rate per attempt, 95% upper bound	1.2e-07

predicts 40% recovery and at twice that. The selected value is then fixed as the default for every policy run, evaluated on the held-out replicates 0–29, and used on the real-read replay. The completion forecast is evaluated with a budget equal to the median coverage-greedy cost, at 30/50/70/85% of it, by expected calibration error (ECE, 10 bins), Brier score against a constant (climatology) forecast [48] and AUC.

E. Real read streams (secondary evidence)

We replay public Illumina iSeq runs of two pools from ENA PRJEB65931 [7], [44]: a 12,000-strand Twist pool after 0/2/7 days of aging or 15/60/90 PCR cycles, and a 12,472-strand Genscript pool (aging 0 d A/B, 2 d, 7 d; PCR 15/30 cycles). Each run emulates one container holding 250 stripes of $(32, 48)$. A fragment counts as recovered when the consensus of its first t reads in a random permutation of the run equals the designed strand, with t drawn from per-strand consensus curves estimated from real reads; decodability is therefore an *oracle* event and the replay is semi-empirical (read orders and counts real, containers emulated by subsetting runs of one synthesis batch). The primary belief is the generic prior; the optional "calibrated" belief now uses only the calibration split (the first 300k reads of each run; evaluation uses the next 900k). 200 read permutations, paired bootstrap.

VI. RESULTS

A. Safety of verified stopping

Table III pools every digital-twin retrieval run for this paper. No retrieval returned a file that differed from the stored one, and no wrong payload passed the inner RS + CRC-16 + address checks. The bounds are those of zero events in the given number of trials; they are statements about this archive geometry, these error rates and this code stack, not about DNA storage in general. Unsuccessful retrievals (one or two failed containers) ended by exhausting the streams and reported failure instead of returning a file.

B. Policy comparison on the digital twin

Table IV and Fig. 2 give the pre-specified comparisons. In the baseline scenario every policy recovered the file in all 300 retrievals. Round-robin with verified stopping used 3.49k reads on average (95% CI 3.37–3.60k). Every model-based rule was costlier on the same syntheses and read orders: Thompson sampling by 7% (4–11%), tail-aware by 13% (9–18%), coverage-greedy by 15% (10–21%), UCB by 15% (11–20%), learn-then-commit by 45% (40–49%), pivotal by 49% (39–60%) and information-greedy by 441% (408–467%). The ordering follows the share of reads spent on the near-dead container: 16% for round-robin, 23% for Thompson sampling, 25–27% for tail-aware, UCB and coverage-greedy, 40–41% for pivotal and learn-then-commit, and 66% for information-greedy. In the homogeneous control (six fresh, independently synthesised containers), where we expected no allocation effect, most model-based rules were 2–4% cheaper than round-robin (coverage-greedy 2.18k vs. 2.26k; information-greedy even at 2.26k): independent syntheses differ enough for mild adaptive gains, consistent with the low-severity end of the sweep below. With DT4DDS's photolithographic synthesis model (reduced design, 30 retrievals per policy) no policy and none of the four static plans recovered the file: every retrieval read the whole 72k-read stream and reported failure, with no wrong output. Like the two-failure case, this is a boundary of the archive design (inner code strength, redundancy and the 60-reads-per-strand stream) under that synthesis model, not a policy result; we did not test deeper streams or a stronger inner code.

C. Why adaptive allocation loses under dropout

The controller models a container by one efficiency multiplier θ_j that scales every fragment's coverage. The near-dead container (two half-lives, 40 PCR cycles, σ=0.5, three copies per strand) instead loses a large fraction of its molecules outright: many of its fragments are never read however long it is sequenced, while the surviving ones are read often. Early batches from it therefore recover fragments at a rate that the model explains with a moderate θ_j and extrapolates to the missing fragments, so coverage-based scores keep predicting gains there. The beta-binomial likelihood (ρ_U=0.1) widens the posterior, which is what calibration asked for, and slows the correction further; information-greedy, which reads the container with the largest posterior variance, never brings that variance below its threshold and spends two thirds of its reads there. A post-hoc run with the binomial likelihood (ρ_U=0, Appendix B) makes coverage-greedy, Thompson sampling and tail-aware 3–4% *cheaper* than round-robin on the same retrievals (pivotal: 4% costlier, against 49%): the better-calibrated posterior is the costlier one. We did not adopt ρ_U=0, because it was not the pre-specified choice and is badly calibrated (Section VI-F). The fix this points to is a dropout-aware container model (a fraction of absent fragments as a second parameter), which we leave for future work.

D. Failed containers and severity sweep

With one failed container (the fresh one, which returns only non-informative reads), 34% of retrievals succeeded under every policy, and success was identical for all policies on every synthesis and read order: whether the file can be recovered

TABLE IV

DIGITAL-TWIN END-TO-END RETRIEVAL (DT4DDS; PRIMARY EVIDENCE). PER SCENARIO AND POLICY: SHARE OF RETRIEVALS WHOSE OUTPUT FILE IS BYTE-IDENTICAL TO THE STORED FILE (NO RETRIEVAL RETURNED A WRONG FILE), MEAN READS PER RETRIEVAL (THOUSANDS; CLUSTER-BOOTSTRAP 95% CI OVER SYNTHESES), THE 0.99-QUANTILE Q99 (THE BUDGET TO RESERVE), AND THE PAIRED RELATIVE DIFFERENCE IN MEAN READS AGAINST ROUND-ROBIN WITH VERIFIED STOPPING ON RETRIEVALS WHERE BOTH SUCCEEDED (95% CI). STATIC PLANS READ FIXED PROPORTIONS UNTIL DECODABLE AND MUST RESERVE THEIR Q99 ON EVERY RETRIEVAL, SO THEIR MEAN COST EQUALS Q99; THEIR PROPORTIONS USE PER-CONTAINER EFFICIENCIES MEASURED IN HINDSIGHT ON THE SAME SCENARIO.

Scenario	Policy	Success	Mean reads (k) [95% CI]	q99 (k)	vs. round-robin, % [95% CI]
Baseline	Round-robin + verified stop	1.00	3.49 [3.37, 3.60]	5.15	
	Coverage-greedy	1.00	4.02 [3.76, 4.30]	9.05	+15 [+10, +21]
	Pivotal	1.00	5.21 [4.76, 5.70]	9.60	+49 [+40, +60]
	Thompson sampling	1.00	3.75 [3.56, 3.95]	6.45	+7 [+4, +11]
	UCB (q=0.9)	1.00	4.02 [3.78, 4.30]	8.15	+15 [+11, +20]
	Learn-then-commit	1.00	5.05 [4.74, 5.31]	7.65	+45 [+40, +49]
	Info-greedy (exp. design)	1.00	18.87 [17.15, 20.37]	37.45	+441 [+407, +470]
	Tail-aware (LCB)	1.00	3.94 [3.72, 4.18]	7.15	+13 [+8, +18]
	Static, uniform (fixed depth)	1.00	5.15 (= q99)	5.15	
	Static, $w \propto e^{0.5}$	1.00	4.90 (= q99)	4.90	
	Static, $w \propto e$ (equal coverage)	1.00	4.80 (= q99)	4.80	
	Static, $w \propto e^2$	1.00	4.90 (= q99)	4.90	
One container failed	Round-robin + verified stop	0.34	20.31^s	–	
	Coverage-greedy	0.34	16.86^s	–	-17 [-23, -9]
	Pivotal	0.34	17.30^s	–	-15 [-21, -6]
	Thompson sampling	0.34	18.43^s	–	-9 [-15, -1]
	UCB (q=0.9)	0.34	15.49^s	–	-24 [-28, -17]
	Learn-then-commit	0.34	21.91^s	–	+8 [+3, +14]
	Info-greedy (exp. design)	0.34	36.68^s	–	+81 [+50, +130]
	Tail-aware (LCB)	0.34	20.63^s	–	+2 [-8, +14]
	Static, uniform (fixed depth)	0.34	–	–	
	Static, $w \propto e^{0.5}$	0.34	–	–	
	Static, $w \propto e$ (equal coverage)	0.34	–	–	
	Static, $w \propto e^2$	0.34	–	–	
Two containers failed	all 12 policies and static plans	0.00	– (stream exhausted, 72k reads)	–	
Homogeneous (control)	Round-robin + verified stop	1.00	2.26 [2.23, 2.29]	2.85	
	Coverage-greedy	1.00	2.18 [2.16, 2.21]	2.70	-3 [-4, -2]
	Pivotal	1.00	2.18 [2.16, 2.20]	2.75	-4 [-5, -3]
	Thompson sampling	1.00	2.19 [2.17, 2.22]	2.70	-3 [-4, -2]
	UCB (q=0.9)	1.00	2.18 [2.16, 2.20]	2.75	-4 [-5, -3]
	Learn-then-commit	1.00	2.19 [2.17, 2.21]	2.75	-3 [-4, -2]
	Info-greedy (exp. design)	1.00	2.25 [2.23, 2.28]	2.90	-0 [-1, +0]
	Tail-aware (LCB)	1.00	2.20 [2.18, 2.23]	2.75	-2 [-4, -1]
	Static, uniform (fixed depth)	1.00	2.85 (= q99)	2.85	
	Static, $w \propto e^{0.5}$	1.00	2.85 (= q99)	2.85	
	Static, $w \propto e$ (equal coverage)	1.00	2.85 (= q99)	2.85	
	Static, $w \propto e^2$	1.00	2.85 (= q99)	2.85	
Photolithographic synthesis[‡]	all 12 policies and static plans	0.00	– (stream exhausted, 72k reads)	–	

30 independent syntheses of all six containers × 10 read orders = 300 retrievals per policy and scenario, except [‡]10 × 3 (protocol deviation, compute). A failed retrieval read the whole stream (72,000 reads) without verifying; [s]: mean over successful retrievals only (a mean or quantile including failures is undefined, "–"). When success < 1, success was identical for every policy on every synthesis and read order, so the paired comparison uses the same retrievals for all policies.

is set by which fragments the remaining five containers lost, i.e. by the archive's redundancy, not by the policy. On the 102 retrievals where all succeeded, adaptive allocation helped, because the failed container is easy to identify and stops being read: UCB used 24% (17–28%) fewer reads than round-robin, coverage-greedy 17% (8–23%), pivotal 15% (6–21%) and Thompson sampling 9% (2–15%); tail-aware was even (+2%, −8 to +14%), learn-then-commit 8% (2–14%) and information-greedy 81% (49–133%) costlier. With two failed containers no policy recovered the file; every retrieval read all 72,000 reads and reported failure. We read both results as boundaries of the *archive design*, not of the policies: with round-robin placement of $(32, 48)$ stripes over six containers, each failed container removes 8 of a stripe's 48 fragments, half of its redundancy of 16. After one failure a stripe tolerates only 8 further lost strands: 18 of the 30 syntheses were unrecoverable under every read order, 7 always recoverable, and 5 depended on the read order (consensus uses at most 24 reads per cluster); after two failures exactly K=32 fragments remain, so a single strand lost by synthesis, aging or PCR in any stripe makes the file unrecoverable. Surviving more failures requires more redundancy or a different placement; no read-allocation policy can compensate.

The severity sweep (Fig. 3) varies only the near-dead container's copies per strand. At 200 and 30 copies, coverage-greedy and Thompson sampling are 2–3% cheaper than round-robin (2.65k vs. 2.72k at 200 copies) and tail-aware is even; at 10 copies the policies are within ±1% except tail-aware (+5%); at 3 and 1 copies all adaptive rules are costlier (at one copy: coverage-greedy 4.69k, Thompson 4.51k, tail-aware 4.47k vs. 4.16k). Success was 1.00 throughout. The point at 3 copies reproduces the baseline design with other seeds (round-

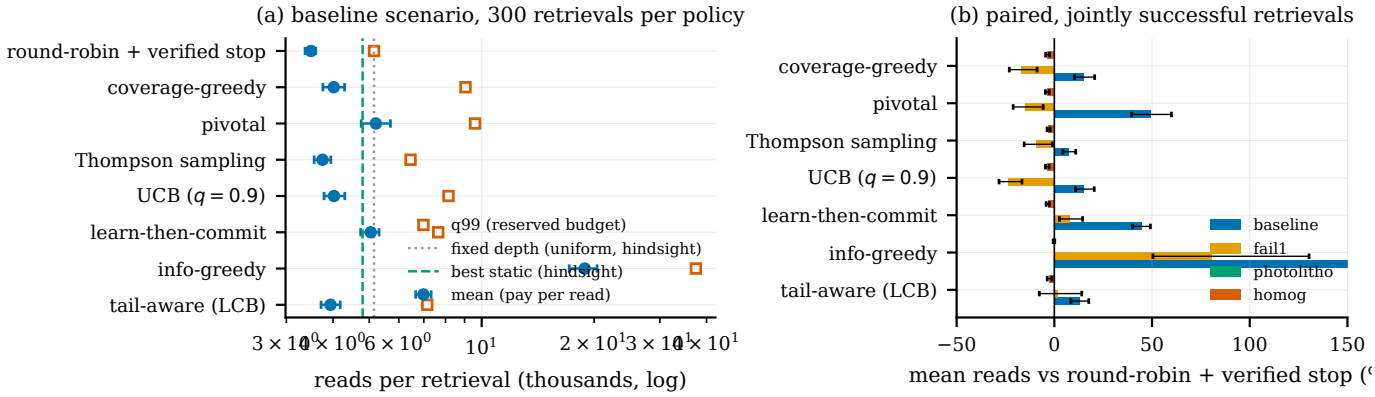


Fig. 2. Digital-twin end-to-end retrieval. (a) Baseline scenario: mean reads (95% cluster-bootstrap CI) and q99 per policy; vertical lines: static plans (whose cost equals their q99). (b) Paired relative cost against round-robin with verified stopping on retrievals where both succeeded (95% CI); negative is cheaper.

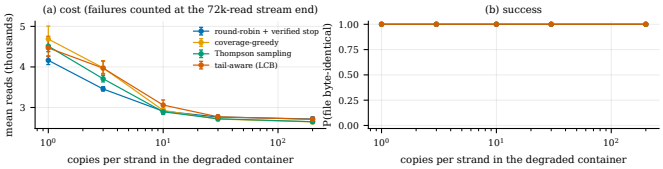


Fig. 3. Severity sweep: the near-dead container's copy number varied from 200 to 1 (300 retrievals per point and policy).

robin 3.46k vs. 3.49k). Adaptive allocation therefore helps only while the model's single multiplier is a fair description of the worst container.

E. Tail and static plans

No allocation rule lowers the 0.99-quantile. In the baseline, round-robin with verified stopping has q99 5.15k; the adaptive rules have 6.45k (Thompson) to 9.6k (pivotal) and the tail-aware rule, designed for the bottleneck stripes, 7.15k. At one copy per strand in the sweep, q99 is 5.65k for round-robin and 10.1k for coverage-greedy. The tail is therefore not fixed, and we reframe the claim: the benefit of this paper's machinery is a lower *mean* cost, which matters to a pay-per-read user. That benefit comes from verified stopping, not from allocation. The static plans have to reserve their q99 on every retrieval: 5.15k for uniform fixed depth and 4.80k for the best hindsight-tuned proportions (α=1), whose weights were measured on the evaluated syntheses and are therefore an optimistic baseline. Round-robin with verified stopping costs 3.49k on average, 27% less than the best static reservation, while needing no knowledge of the containers. A user who must reserve capacity gains nothing from stopping early unless unused capacity is refunded; for that user the q99 of round-robin (5.15k) equals the uniform static budget, and the tuned static plan is the better choice. In the homogeneous control the static plans reserve 2.85k against a round-robin mean of 2.26k (-21%). With failed containers no static plan can be sized: it either reads the whole stream or fails, and its q99 is the full 72k reads.

F. Calibration and completion forecast

On the ten selection replicates, 90% credible-interval coverage of θ^* rose with ρ_U from 0.45 (binomial) through 0.63, 0.73 and 0.80 to 0.87 at ρ_U=0.1, the edge of the grid; we kept the pre-specified grid and selected 0.1. On the 30 held-out replicates the selected value gave the same overall coverage, 0.87 (0.84 at the earlier and 0.89 at the later read count), and 1.00 for five of the six containers but 0.20 for the near-dead one, whose target the generic model cannot fit (Fig. 4). With the binomial likelihood held-out coverage is 0.54.

The completion forecast was evaluated on 1,127 checkpoints from 300 baseline retrievals with a budget of 3.75k reads (the median coverage-greedy cost); 52% of retrievals finished within it. Both forecasts were over-confident (mean forecast 0.86 vs. 0.52 observed at 30% of the budget). The posterior-simulation forecast was better than the analytic one on every metric (ECE 0.29 vs. 0.34; Brier 0.30, 95% CI 0.27–0.34, vs. 0.34, 0.29–0.38; AUC 0.73, 0.69–0.78, vs. 0.69, 0.65–0.73; cluster bootstrap), but both Brier scores are worse than the 0.25 of a constant forecast at the observed base rate. The forecasts rank retrievals usefully late in a retrieval (AUC 0.82 at 85% of the budget) but should not be used as probabilities. Futility stopping, which would rely on them, stays disabled.

G. Secondary evidence: real read streams

Table V gives the leak-free replay with the generic prior. On the Twist pool, round-robin with verified stopping used 58.6k reads on average against the 65.7k budget of the hindsight-tuned static plan (-11%), and the model-based rules used 56.6–57.2k, 2–3% fewer than round-robin; coverage-greedy was 9.0k (95% CI 8.5–19.3k) below the hindsight static budget. Information-greedy reduced to coverage-greedy after the first probes (identical results), and the leak-free calibrated belief changed coverage-greedy by -0.2%. As on the digital twin, no rule lowered q99 materially: 65.6–67.2k for the model-based rules against the 65.7k static budget. Stopping supplies most of the gain (about 7.1k of 9.0k reads). On the Genscript pool, whose PCR-30-cycle container is near-dead, verified stopping with round-robin used 637k reads against

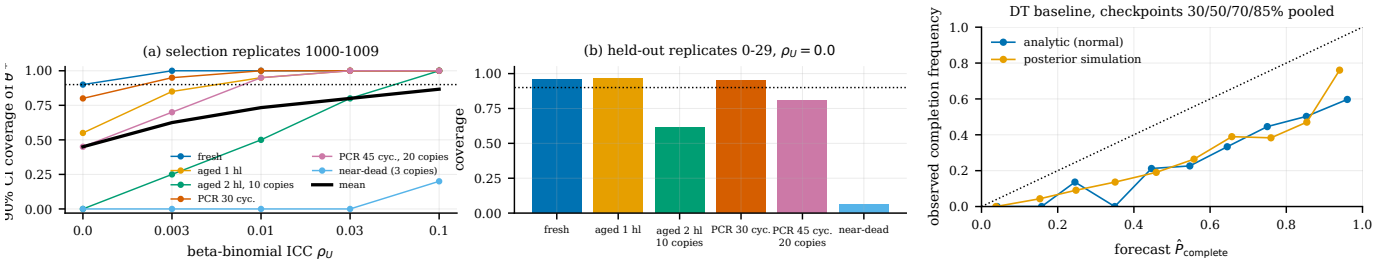


Fig. 4. Calibration on the digital twin. Left: 90% credible-interval coverage of θ^* per container on the selection replicates for each ρ_U, and on held-out replicates at the selected ρ_U. Right: reliability of the completion forecast (analytic vs. posterior simulation).

TABLE V
SECONDARY EVIDENCE: SEMI-EMPIRICAL REPLAY OF REAL ILLUMINA READ STREAMS (ENA PRJEB65931), GENERIC PRIOR, NO EVALUATION-SPLIT STATISTICS. MEAN READS (THOUSANDS) AND Q99; Δ: MEAN MINUS THE HINDSIGHT-TUNED STATIC BUDGET (PAIRED BOOTSTRAP 95% CI). DECODABILITY IS AN ORACLE EVENT HERE (CONSENSUS EQUALS THE DESIGNED STRAND), SO THIS IS *not* EXACT RETRIEVAL.

Pool	Policy	Mean	q99	Δ vs. static [95% CI]
Twist	Static, uniform (hindsight budget)	68.1	68.1	
	Static, hindsight-tuned	65.7	65.7	
	Round-robin + verified stop	58.6	68.4	-7.1 [-17.4, -6.5]
	Coverage-greedy	56.7	66.0	-9.0 [-19.3, -8.5]
	Pivotal	56.6	67.2	-9.1 [-19.4, -8.6]
	Thompson sampling	56.9	66.0	-8.9 [-19.1, -8.4]
	UCB	56.7	66.6	-9.0 [-19.2, -8.5]
	Learn-then-commit	57.2	66.6	-8.5 [-18.8, -8.0]
	Info-greedy	56.7	66.0	-9.0 [-19.3, -8.5]
	Tail-aware (LCB)	56.7	65.6	-9.0 [-19.3, -8.5]
	Coverage-greedy, leak-free calibrated	56.6	67.6	-9.1 [-19.4, -8.6]
Genscript	Static, uniform (hindsight budget)	852	852	
	Static, hindsight-tuned	631	631	
	Round-robin + verified stop	637	853	+6.0 [-57.4, +15.1]
	Coverage-greedy	639	820	+8.2 [-54.9, +17.2]
	Pivotal	639	820	+8.2 [-54.9, +17.2]
	Thompson sampling	654	831	+23.4 [-39.4, +31.9]
	UCB	637	830	+6.0 [-57.8, +14.9]
	Learn-then-commit	651	843	+19.7 [-42.8, +29.2]
	Info-greedy	not completed (Appendix B)		
	Tail-aware (LCB)	644	827	+12.9 [-51.0, +21.6]
	Coverage-greedy, leak-free calibrated	533	676	-97.9 [-161.6, -90.0]

852k for fixed depth (−25%), but 1% more than the 631k of the hindsight-tuned static plan, which knows in advance to give that container 2.7% of its reads (paired difference +6.0k, 95% CI −57.4 to +15.1k). Every model-based rule with the generic prior was 0–3% costlier than round-robin (UCB 637k, coverage-greedy and pivotal 639k, tail-aware 644k, learn-then-commit 651k, Thompson sampling 654k), and, as on the digital twin, each gave the near-dead container more reads than round-robin did (19–20% against 17%). The exception is the leak-free calibrated belief, fitted on calibration reads of the same containers that are disjoint from the evaluation reads: coverage-greedy with it gave the near-dead container 3.4% of its reads and used 533k (−16% vs. round-robin; −97.9k, 95% CI −161.6 to −90.0k, vs. the hindsight static budget). Allocation therefore pays when the weak container is identified *before* retrieval, not when the controller must learn it online. q99 was 820–853k for the generic-prior rules and 676k with the calibrated belief, against the 631k static budget. The information-greedy run on this pool did not complete in time (Appendix B).

H. Supporting channel result (real nanopore reads)

On two independent nanopore flowcells of DNAformer reads [40], [49], a strand-specific error propensity persists across flowcells (r=0.57; null with matched contexts 0.002), and a hierarchical beta-binomial error model fitted on one flowcell predicts consensus success on held-out strands of the other with mean absolute error 0.083, against 0.181 for an i.i.d. model. This motivates over-dispersed models but does not enter the controller.

VII. DISCUSSION

What helps, and whom. The robust gain in our experiments is verified stopping: reading until every stripe, its CRC-32 and the file hash verify, and then stopping. It needs no model, never returned a wrong file, and costs a pay-per-read user 27% less on average than the best hindsight-tuned static plan in the baseline digital twin. It does not reduce the reads a user must reserve. Choosing *which* container to read with a posterior model helped only where the model fits: an outright failure, which the posterior detects within a few batches, or mild heterogeneity. With a near-dead container, every model-based rule we tried, including standard bandit and experimental-design rules, was costlier than round-robin.

Calibration versus cost. The likelihood that gave the best-calibrated intervals (ρ_U=0.1) also gave the costlier allocations, and the binomial likelihood, whose intervals covered only 54%, gave the cheaper ones. Calibrating the uncertainty of a misspecified model does not make its decisions better; the misspecification (one multiplier for a container with dropout) is the problem to fix. We therefore present the controller as online-adaptive, not self-calibrating, and do not recommend its forecasts as probabilities.

Digital twin versus replay. The two settings agree that stopping is the main saving and that no rule lowers q99; on allocation with the generic prior they agree where the conditions match. On the Twist pool, whose containers are runs of one synthesis batch with no near-dead member (the mild-heterogeneity end of the sweep), allocation saved 2–3%; on the Genscript pool, which has a near-dead container, it cost 0–3% and over-read that container, as on the digital twin. Only

Practical recommendation (multi-container retrieval).

1) Read the containers in turn and **stop on verification** (every stripe, its CRC-32 and the file hash). This is the robust saving and needs no model.
2) **Switch a container off** once it is clearly failed (no verified fragment after a probe); here posterior rules helped (up to -24%).
3) Do **not** rely on model-based allocation when a container may be near-dead: every rule we tried read more than round-robin. Use it only with a container model that represents dropout, or when calibration reads already identify the weak container (-16% on the Genscript replay), and check it against round-robin on a digital twin first.
4) If capacity must be **reserved**, budget the q99 of round-robin; no rule lowered it.
5) Size the archive's redundancy for the failures you must survive: two failed containers out of six, or a noisier synthesis, made the file unrecoverable whatever the policy.

Fig. 5. Practical recommendation derived from Section VI.

prior knowledge of the weak container, through calibration reads, turned allocation into a large saving.

Practical recommendation. The box in Fig. 5 summarises what our results support for archives like ours.

What we would test with wet-lab containers. With physically separate containers we would test (X) whether real containers show the near-dead, dropout-dominated behaviour that made adaptive allocation lose here, by synthesising one design in several independent batches, aging and amplifying them differently, and sequencing each container to saturation to measure its fraction of absent strands; (Y) whether verified stopping keeps its saving and its safety on real sequencing runs with our own inner code, by streaming reads from a nanopore or a split Illumina run through the same decoder and counting wrong outputs and inner-code false accepts against the known file; and (Z) what real container failures look like and how often they occur, by tracking a set of containers over accelerated aging, which would calibrate the failure prior and the redundancy needed per container.

VIII. Limitations and Threats to Validity

Table VI lists the main threats. *External validity*: every result is computational. The digital twin uses DT4DDS's default parameters and one archive geometry (1,200 strands, $(32, 48)$, six containers), and its failure mode is simulated (a failed container returns non-informative reads). The replay uses real reads, but decodability is an oracle event and the "containers" are runs of one synthesis batch. *Internal validity*: ρ_U was selected with prior sd $s{=}1$ and used with $s{=}0.5$; the selected value lies at the edge of the pre-specified grid; the photolithographic scenario uses a reduced design; and one code fix (static allocator) was made after freezing (Appendix B). The static baselines use efficiencies measured in hindsight on the evaluated syntheses, which favours them. *Construct validity*: read counts are a proxy for cost; we do not model run set-up costs, which favour fewer, larger batches. *Statistical*: wrong-output and false-accept bounds come from zero events and scale with the number of trials; the comparisons are paired but within one archive design. We prove no optimality or regret result.

TABLE VI
Assumptions, limitations and their consequence.

Limitation	Consequence
Computational only (digital twin + replay)	no wet-lab confirmation; DT4DDS fidelity bounds external validity
DT4DDS default parameters, one archive geometry	conclusions may change with other codes, strand lengths, error regimes
Failures simulated (junk reads)	real failure modes and rates unknown; failure budget uncalibrated
Single efficiency multiplier θ_j	cannot represent dropout; adaptive allocators over-read near-dead containers
ρ_U at the edge of its grid	selection coverage $0.87 < 0.90$; grid not extended post hoc
ρ_U selected with $s{=}1$, runs use $s{=}0.5$	selection and use not perfectly aligned
Calibration-optimal vs. cost-optimal ρ_U differ	better-calibrated posterior made allocation costlier (post-hoc)
Gains in the mean only	no allocator, including the tail-aware one, lowers q99
Replay is semi-empirical	oracle decodability; containers are aliquots of one batch
Abstract read costs, no setup cost	favours small batches; Illumina run economics not modelled
Reduced design for photolithographic scenario	30 instead of 300 retrievals per policy
No theory (regret, optimality)	empirical claims only

IX. Reproducibility and Availability

All code, configuration and result files are in the accompanying repository and artifact archive. The digital-twin experiments are driven by `experiments/dt_experiments.py` (subcommands `gen`, `run`, `static`, `calib`, `calib_eval`, `forecast`, `posthoc_rho0`); the replay by `experiments/paper_experiments.py replay_v3`; statistics by `dt_analysis.py`; tables and figures by `make_tables.py` and `make_dt_figures.py`, which read only committed result files (no number in the tables is entered by hand). Every random draw is seeded by CRC-32 of a string naming the scenario, replicate, container, read order and policy, so results do not depend on process scheduling or worker count. Re-executing 24 randomly sampled baseline retrievals in a fresh single-process run, in reverse order, reproduced every recorded field (reads, per-container reads, rounds, decode attempts, status) exactly (`rerun_check.py`). The protocol (`dt_protocol.yaml`) with its deviation log, the DT4DDS version (1.1.0), Python and package versions are in the artifact. The public data are ENA PRJEB65931 (Illumina) and the DNAformer nanopore reads. Table VII maps each claim to its experiment and result file.

X. Conclusion

On a digital twin with independently synthesised containers, verified stopping retrieved the file without a single wrong output and cut the mean reads per retrieval by 27% against the best static reservation; adaptive container selection added nothing when a container was near-dead and helped (by up to

TABLE VII

CLAIM-TO-EVIDENCE SUMMARY (FULL TABLE: `evidence/evidence_traceability.csv`). DT: DT4DDS DIGITAL TWIN, FULL DECODE CHAIN, NO ORACLE. REPLAY: REAL READS, ORACLE DECODABILITY. NO RESULT IS WET-LAB EVIDENCE.

Claim	Command (`dt_experiments.py` unless noted)	Result file (`results/`)	Evidence
No wrong output, no false accept (Table III)	all DT runs	`dt_*.csv`	DT
Verified stop: -27% mean vs. best static reservation	`run`/`static baseline`	`dt_baseline.csv`, `dt_baseline_static.csv`	DT
Model-based rules costlier than round-robin (baseline)	`run baseline`	`dt_summary.csv`	DT, negative
Gains after outright failure (UCB -24%); success set by redundancy	`run fail1`, `run fail2`	`dt_fail{1,2}{,_static}.csv`	DT, simulated failures
Crossover with severity of the worst container	`run sweep_cov*`	`dt_sweep_cov*.csv`	DT
No rule lowers q99	all DT runs	`dt_summary.csv`	DT, negative
ρ_U selection and held-out coverage	`calib`, `calib_eval`	`dt_calibration_{select, eval}_baseline.csv`	DT, partly negative
Forecasts over-confident, worse than climatology	`forecast baseline`	`dt_forecast_baseline.csv`	DT, negative
Binomial likelihood cheaper (post hoc)	`posthoc_rho0`	`dt_*_posthoc_rho0.csv`	DT, post hoc
Homogeneous control; photolithographic synthesis	`run homog`, `run photolitho`	`dt_homog{,_static}.csv`, `dt_photolitho{,_static}.csv`	DT (photolitho reduced)
Leak-free real-read replay	`paper_experiments.py replay_v3`	`replay_v3{,_genscript}.csv`	Replay
Leaky first analysis (upper bound only)	`real_data.py policies`	`realdata_*policies*_v2.csv`	superseded
Correlated nanopore errors (MAE 0.083 vs. 0.181)	`q3_correlated_errors.py`	`q3_summary.json`	real nanopore reads
Regret bounds; wet-lab containers; real failure rates	–	–	not attempted

24%) only after an outright failure or under mild heterogeneity. No rule lowered the 0.99-quantile, and the controller's forecasts are over-confident. The results are computational, for one archive design and one simulator; their value is a reproducible, leak-free baseline and a clear account of where online adaptation fails. A container model with dropout, wet-lab containers and a real failure distribution are the next steps.

APPENDIX A
UPPER BOUND WITH EVALUATION LEAKAGE

Our first analysis of the real read streams (Table VIII) used a "calibrated" belief whose per-strand consensus curves were pooled over reads that included the evaluation split. The controller therefore had information about the very reads it was evaluated on, and the static plans were tuned on them too. We keep the table only as an *upper bound with evaluation leakage*; no claim in the main text uses it. In the leak-free re-analysis (Table V) the primary belief is the generic prior, and the optional calibrated belief uses only the first 300k reads of each run while evaluation uses the next 900k. On Twist, the leaky analysis put coverage-greedy at 55.7k reads and the oracle-tuned static plan at 63.1k; the leak-free analysis gives 56.7k and 65.7k. The leak therefore flattered the adaptive policy by about 1k reads and the static baseline by about 2.6k, and the leak-free gap is larger, not smaller.

APPENDIX B
PROTOCOL DEVIATIONS AND POST-HOC ANALYSES

Deviations. The frozen protocol and every change made after freezing are in `dt_protocol.yaml`. (i) The planned CustomArray scenario was replaced, before any policy run, by DT4DDS's photolithographic synthesis model, because the CustomArray settings carry a 117-nt position-bias table that cannot be applied to 136-nt strands. (ii) We suspected from commit timestamps that the first Twist replay had run with $\rho_U{=}0$ and re-ran it; a direct test showed that it had used the frozen $\rho_U{=}0.1$, and the re-run reproduced it exactly, so nothing changed. (iii) One cached homogeneous replicate was truncated by an interrupted job; it was regenerated from its seed and the scenario re-run in full. (iv) The photolithographic scenario was run with 10 syntheses $\times$ 3 read orders for compute reasons; the first 50 retrievals of the full run were discarded unseen. (v) The static allocator crashed when only zero-weight (failed) containers remained; it now reads them out, which can only affect failed retrievals of the failure scenarios. The ρ_U rule was applied as frozen even though the selected value lies at the edge of the grid. (vi) The sequential Genscript replay finished information-greedy after about six hours but then crashed while computing its paired difference, because some trials never completed (infinite cost); nothing was saved. We fixed the bug, ran the other Genscript conditions in parallel processes with the same per-condition seeds (they reproduced every value logged by the sequential run), and re-started information-greedy; it had not finished when this version was built, so Table V has no information-greedy row for Genscript. The fact that some of its trials did not complete within the read stream is the only information-greedy result we report for this pool.

Post-hoc analyses (not pre-specified). With the binomial likelihood ($\rho_U{=}0$), on the same 300 baseline retrievals, coverage-greedy used 3.34k reads (-4.4% vs. round-robin, against $+15.1\%$ at $\rho_U{=}0.1$), Thompson sampling 3.37k (-3.5%, against $+7.4\%$), tail-aware 3.37k (-3.5%, against $+12.9\%$) and pivotal 3.64k ($+4.4\%$, against $+49.3\%$); q99 was 4.95–5.70k, still not below round-robin's 5.15k. With one failed container the binomial likelihood helped tail-aware (-25% vs. $+2\%$) and Thompson sampling (-20% vs. -9%) and hurt coverage-greedy and pivotal slightly (-15% vs. -17%; -8% vs. -15%). Success and safety were unchanged (no wrong output). On the Twist replay, the binomial likelihood made coverage-greedy and Thompson sampling costlier (57.6k each, against 56.7k and 56.9k at $\rho_U{=}0.1$), the opposite of the digital-twin result: the two settings disagree on which

TABLE VIII

UPPER BOUND WITH EVALUATION LEAKAGE (SUPERSEDED FIRST ANALYSIS, KEPT FOR TRANSPARENCY). REPLAY OF REAL READ STREAMS WITH "CALIBRATED" BELIEFS WHOSE CONSENSUS CURVES WERE POOLED FROM *evaluation*-SPLIT READS (APPENDIX A); THOUSANDS OF READS AT 0.99. THESE NUMBERS ARE NOT USED FOR ANY CLAIM IN THE MAIN TEXT. STATIC PLANS PAY THEIR BUDGET ON EVERY RETRIEVAL; FOR ADAPTIVE POLICIES THE 0.99-QUANTILE Q99 IS SHOWN SEPARATELY. Δ: ADAPTIVE (PIVOTAL) MINUS ORACLE-TUNED STATIC, PAIRED (95% CI).

Pool	Placement	Fixed depth	Oracle-tuned static	Adaptive stop, round-robin	Adaptive, cov.-greedy	Adaptive, pivotal	q99 of pivotal	Δ (95% CI)
Twist	round-robin (6)	67.9	63.1	57.8	55.7	55.8	63.2	-7.3 (-12.1, -7.0)
Twist	reliability-weighted	65.4	61.9	56.0	54.2	54.1	61.8	-7.9 (-13.3, -7.6)
Twist	tiered (4)	61.9	53.1	51.3	46.4	46.2	53.0	-6.9 (-19.1, -6.2)
Twist	single best	53.9	53.9	44.8	44.8	44.8	54.0	-9.1 (-11.0, -6.7)
Genscript	round-robin (6)	810.4	617.9	639.9	532.1	531.9	694.2	-86.0 (-127.1, -78.8)
Genscript	single best†	279.4	279.4	219.0	219.0	219.0	279.6	-60.4 (-70.5, -54.2)

200 read permutations, except †100. All policies completed in every trial. The single-best container is chosen with the calibrated belief, which contains the evaluation-split leak of Appendix A. Superseded first-run Genscript numbers (computed before the per-strand consensus cache was regenerated) are kept in `results/realdata_genscript_policies_{roundrobin,part2}.csv`.

likelihood allocates better.

ACKNOWLEDGMENT

We thank the authors of the public datasets and of DT4DDS (ENA PRJEB65931; DNAformer; Srinivasavaradhan *et al.*) for releasing reads, designs and software.

REFERENCES

[1] G. M. Church, Y. Gao, and S. Kosuri, "Next-generation digital information storage in DNA," *Science*, vol. 337, no. 6102, p. 1628, 2012.

[2] N. Goldman, P. Bertone, S. Chen, C. Dessimoz, E. M. LeProust, B. Sipos, and E. Birney, "Towards practical, high-capacity, low-maintenance information storage in synthesized DNA," *Nature*, vol. 494, no. 7435, pp. 77–80, 2013.

[3] R. N. Grass, R. Heckel, M. Puddu, D. Paunescu, and W. J. Stark, "Robust chemical preservation of digital information on DNA in silica with error-correcting codes," *Angewandte Chemie International Edition*, vol. 54, no. 8, pp. 2552–2555, 2015.

[4] Y. Erlich and D. Zielinski, "DNA fountain enables a robust and efficient storage architecture," *Science*, vol. 355, no. 6328, pp. 950–954, 2017.

[5] L. Organick, S. D. Ang, Y.-J. Chen, R. Lopez, S. Yekhanin, K. Makarychev, M. Z. Racz, G. Kamath, P. Gopalan, B. Nguyen, C. N. Takahashi, S. Newman, H.-Y. Parker, C. Rashtchian, K. Stewart, G. Gupta, R. Carlson, J. Mulligan, D. Carmean, G. Seelig, L. Ceze, and K. Strauss, "Random access in large-scale DNA data storage," *Nature Biotechnology*, vol. 36, no. 3, pp. 242–248, 2018.

[6] R. Heckel, G. Mikutis, and R. N. Grass, "A characterization of the DNA data storage channel," *Scientific Reports*, vol. 9, no. 1, p. 9663, 2019.

[7] A. L. Gimpel, W. J. Stark, R. Heckel, and R. N. Grass, "A digital twin for DNA data storage based on comprehensive quantification of errors and biases," *Nature Communications*, vol. 14, no. 1, p. 6026, 2023.

[8] D. Bar-Lev, O. Sabary, R. Gabrys, and E. Yaakobi, "Cover your bases: How to minimize the sequencing coverage in DNA storage systems," *IEEE Transactions on Information Theory*, vol. 71, no. 1, pp. 192–218, 2025.

[9] A. Gruica, D. Bar-Lev, A. Ravagnani, and E. Yaakobi, "A combinatorial perspective on random access efficiency for DNA storage," in *Proc. IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2024, pp. 675–680.

[10] J. Bornholt, R. Lopez, D. M. Carmean, L. Ceze, G. Seelig, and K. Strauss, "A DNA-based archival storage system," in *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2016, pp. 637–649.

[11] X. Kong, O. Elishco, C. Wang, and T. M. Duman, "The coverage depth problem in distributed DNA data storage," arXiv:2610.02931 [cs.IT], 2026, preprint.

[12] A. Levy, R. Con, E. Yaakobi, and H. M. Kiah, "Expected recovery time in DNA-based distributed storage systems," arXiv:2602.07601 [cs.IT], 2026, preprint.

[13] R. Lopez, Y.-J. Chen, S. Dumas Ang, S. Yekhanin, K. Makarychev, M. Z. Racz, G. Seelig, K. Strauss, and L. Ceze, "DNA assembly for nanopore data storage readout," *Nature Communications*, vol. 10, no. 1, p. 2933, 2019.

[14] W. R. Thompson, "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples," *Biometrika*, vol. 25, no. 3-4, pp. 285–294, 1933.

[15] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, no. 2–3, pp. 235–256, 2002.

[16] A. Garivier, T. Lattimore, and E. Kaufmann, "On explore-then-commit strategies," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, 2016.

[17] K. Chaloner and I. Verdinelli, "Bayesian experimental design: A review," *Statistical Science*, vol. 10, no. 3, pp. 273–304, 1995.

[18] P. L. Antkowiak, J. Lietard, M. Z. Darestani, M. M. Somoza, W. J. Stark, R. Heckel, and R. N. Grass, "Low cost DNA data storage using photolithographic synthesis and advanced information reconstruction and error correction," *Nature Communications*, vol. 11, no. 1, p. 5345, 2020.

[19] W. H. Press, J. A. Hawkins, S. K. Jones, J. M. Schaub, and I. J. Finkelstein, "HEDGES error-correcting code for DNA storage corrects indels and allows sequence constraints," *Proceedings of the National Academy of Sciences*, vol. 117, no. 31, pp. 18 489–18 496, 2020.

[20] Y.-J. Chen, C. N. Takahashi, L. Organick, C. Bee, S. D. Ang, P. Weiss, B. Peck, G. Seelig, L. Ceze, and K. Strauss, "Quantifying molecular bias in DNA data storage," *Nature Communications*, vol. 11, no. 1, p. 3264, 2020.

[21] I. Shomorony and R. Heckel, "DNA-based storage: Models and fundamental limits," *IEEE Transactions on Information Theory*, vol. 67, no. 6, pp. 3675–3689, 2021.

[22] N. Weinberger and N. Merhav, "The DNA storage channel: Capacity and error probability bounds," *IEEE Transactions on Information Theory*, vol. 68, no. 9, pp. 5657–5700, 2022.

[23] A. Lenz, P. H. Siegel, A. Wachter-Zeh, and E. Yaakobi, "Coding over sets for DNA storage," *IEEE Transactions on Information Theory*, vol. 66, no. 4, pp. 2331–2351, 2020.

[24] R. Cao and X. Chen, "Optimizing sequencing coverage depth in DNA storage: Insights from DNA storage data," in *Proc. IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2025.

[25] S. Kas Hanna, "On the reliability of information retrieval from MDS coded data in DNA storage," *IEEE Transactions on Molecular, Biological, and Multi-Scale Communications*, vol. 12, pp. 585–598, 2026.

[26] M. Loose, S. Malla, and M. Stout, "Real-time selective sequencing using nanopore technology," *Nature Methods*, vol. 13, no. 9, pp. 751–754, 2016.

[27] A. Payne, N. Holmes, T. Clarke, R. Munro, B. J. Debebe, and M. Loose, "Readfish enables targeted nanopore sequencing of gigabase-sized genomes," *Nature Biotechnology*, vol. 39, no. 4, pp. 442–450, 2021.

[28] L. Weilguny, N. De Maio, R. Munro, C. Manser, E. Birney, M. Loose, and N. Goldman, "Dynamic, adaptive sampling during nanopore se-

quencing using Bayesian experimental design," *Nature Biotechnology*, vol. 41, no. 7, pp. 1018–1025, 2023.

[29] D. J. Russo, B. Van Roy, A. Kazerouni, I. Osband, and Z. Wen, "A tutorial on Thompson sampling," *Foundations and Trends in Machine Learning*, vol. 11, no. 1, pp. 1–96, 2018.

[30] T. Lattimore and C. Szepesvári, *Bandit Algorithms*. Cambridge University Press, 2020.

[31] S. Weil, S. Brandt, E. Miller, and C. Maltzahn, "CRUSH: Controlled, scalable, decentralized placement of replicated data," in *Proceedings of the 2006 ACM/IEEE Conference on Supercomputing (SC'06)*. IEEE, 2006, p. 31.

[32] A. Cidon, S. Rumble, R. Stutsman, S. Katti, J. Ousterhout, and M. Rosenblum, "Copysets: Reducing the frequency of data loss in cloud storage," in *2013 USENIX Annual Technical Conference (USENIX ATC '13)*. San Jose, CA, USA: USENIX Association, 2013, pp. 37–48. [Online]. Available: https://www.usenix.org/conference/atc13/technical-sessions/presentation/cidon

[33] D. Ford, F. Labelle, F. I. Popovici, M. Stokely, V.-A. Truong, L. Barroso, C. Grimes, and S. Quinlan, "Availability in globally distributed storage systems," in *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI '10)*. Vancouver, BC, Canada: USENIX Association, 2010. [Online]. Available: https://www.usenix.org/conference/osdi10/availability-globally-distributed-storage-systems

[34] S. Nath, H. Yu, P. B. Gibbons, and S. Seshan, "Subtleties in tolerating correlated failures in wide-area storage systems," in *3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. San Jose, CA, USA: USENIX Association, 2006. [Online]. Available: https://www.usenix.org/conference/nsdi-06/subtleties-tolerating-correlated-failures-wide-area-storage-systems

[35] D. Leong, A. G. Dimakis, and T. Ho, "Distributed storage allocations," *IEEE Transactions on Information Theory*, vol. 58, no. 7, pp. 4733–4752, 2012.

[36] T. Batu, S. Kannan, S. Khanna, and A. McGregor, "Reconstructing strings from random traces," in *Proceedings of the 15th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, New Orleans, LA, USA, 2004, pp. 910–918.

[37] S. R. Srinivasavaradhan, S. Gopi, H. D. Pfister, and S. Yekhanin, "Trellis BMA: Coded trace reconstruction on IDS channels for DNA storage," in *Proc. IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2021, pp. 2453–2458.

[38] O. Sabary, A. Yucovich, G. Shapira, and E. Yaakobi, "Reconstruction algorithms for DNA-storage systems," *Scientific Reports*, vol. 14, no. 1, p. 1951, 2024.

[39] C. Rashtchian, K. Makarychev, M. Racz, S. Ang, D. Jevdjic, S. Yekhanin, L. Ceze, and K. Strauss, "Clustering billions of reads for DNA data storage," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/hash/ab7314887865c4265e896c6e209d1cd6-Abstract.html

[40] D. Bar-Lev, I. Orr, O. Sabary, T. Etzion, and E. Yaakobi, "Scalable and robust DNA-based storage via coding theory and deep learning," *Nature Machine Intelligence*, vol. 7, no. 4, pp. 639–649, 2025.

[41] C. Delahaye and J. Nicolas, "Sequencing DNA with nanopores: Troubles and biases," *PLOS ONE*, vol. 16, no. 10, p. e0257521, 2021.

[42] R. R. Wick, L. M. Judd, and K. E. Holt, "Performance of neural network basecalling tools for Oxford Nanopore sequencing," *Genome Biology*, vol. 20, no. 1, p. 129, 2019.

[43] S. Bubeck, D. Ernst, and A. Garivier, "Optimal discovery with probabilistic expert advice: Finite time analysis and macroscopic optimality," *Journal of Machine Learning Research*, vol. 14, no. 17, pp. 601–623, 2013. [Online]. Available: https://www.jmlr.org/papers/v14/bubeck13a.html

[44] FML, ETH Zurich, "DT4DDS – characterization of errors and biases in DNA data storage," European Nucleotide Archive, study PRJEB65931, 2023, sequencing data of Gimpel et al., Nat. Commun. 14:6026 (2023); first public 2023-09-08. [Online]. Available: https://www.ebi.ac.uk/ena/browser/view/PRJEB65931

[45] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300–304, 1960.

[46] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Springer US, 1993.

[47] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.

[48] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.

[49] Technion – Israel Institute of Technology, "Binned Nanopore/Illumina reads from DNAformer, split by the data type," Zenodo dataset, 2025. [Online]. Available: https://doi.org/10.5281/zenodo.17473983